

Cómo se Ejecuta el Código

Ingeniería de Sistemas — Práctica

Universidad Austral — 2026

Lo que vamos a construir

PrintScript: un lenguaje de programación propio.

```
let x: number = 5;  
let saludo: string = "hola";  
println(saludo);
```

Y todo el *tooling* alrededor: ***lexer***, ***parser***, ***interpreter***, ***formatter***, ***linter*** y una **CLI**.

La pregunta

Cuando ejecutás esto:

```
println(2 + 3 * 4);
```

...la máquina no entiende nada de eso. Solo ve una secuencia de caracteres.

¿Cómo se llega de **texto** a **comportamiento**?

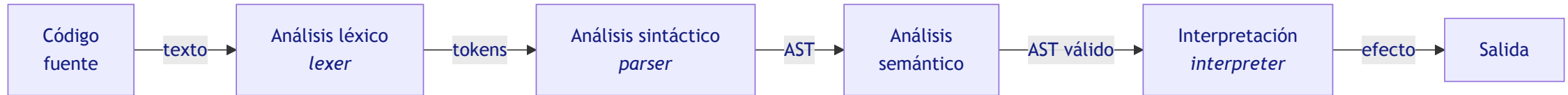
Compile vs interpret

Compilador — traduce el programa entero a otro lenguaje (máquina, *bytecode*) y después se ejecuta el resultado.

Intérprete — recorre el programa y lo va ejecutando directamente.

Las etapas de análisis del texto son **las mismas** en los dos casos.

El *pipeline*



Cada etapa tiene **una entrada** y **una salida**, y no sabe nada de las otras.

Lexer

de caracteres a *tokens*

Qué hace el *lexer*

Agrupar caracteres en las **unidades mínimas con significado** del lenguaje.

```
let x: number = 5;
```



```
LET IDENTIFIER("x") COLON TYPE("number")  
ASSIGN NUMBER_LITERAL("5") SEMICOLON
```

Qué lleva un *token*

```
record Token(  
    TokenType type,    // LET, IDENTIFIER, NUMBER_LITERAL, ...  
    String value,      // "x", "5", "hola"  
    Position start,    // línea y columna  
    Position end  
) {}
```

La **posición** parece un detalle. No lo es: sin ella no hay mensajes de error útiles ni *linter* que sirva.

El *lexer* no valida

```
let 5 = "hola" number;
```

Esto tokeniza **perfecto**. Es una secuencia válida de *tokens*.

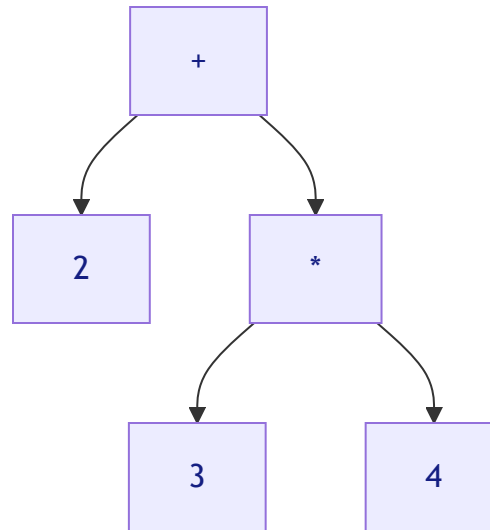
Que no tenga ningún sentido es problema del **parser**.

Parser

de *tokens* a árbol

Por qué un árbol

Para $2 + 3 * 4$, la lista de *tokens* es plana, pero el significado **no** lo es.

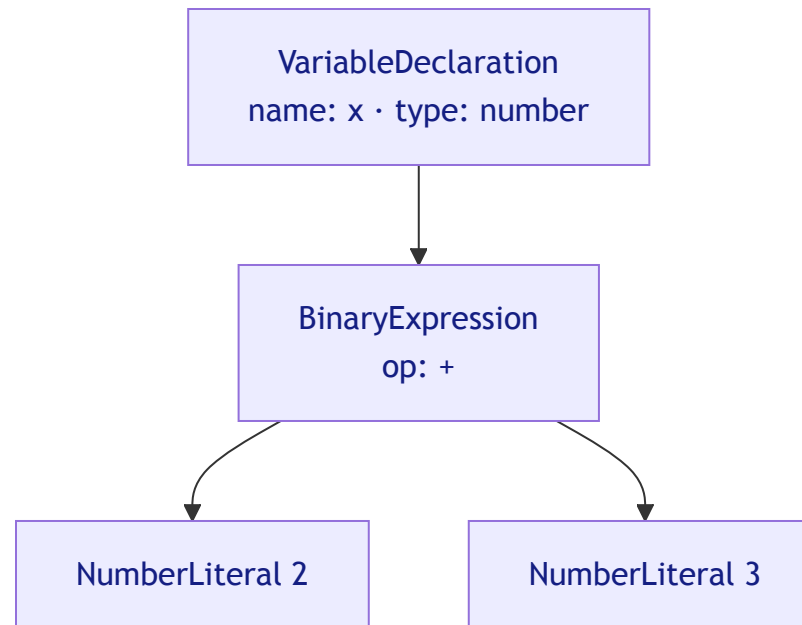


El árbol codifica la **precedencia**: el resultado es 14, no 20.

AST — *Abstract Syntax Tree*

Abstract porque tira lo que ya no hace falta: paréntesis, punto y coma, espacios. Esa información ya está en la **forma** del árbol — acá,

```
let x: number = 2 + 3;
```



Un pedacito de gramática

```
program      = { statement } ;
statement    = declaration | assignment | call ;
declaration  = "let" , identifier , ":" , type , [ "=" , expression ] , ";" ;
expression   = term , { ( "+" | "-" ) , term } ;
term         = factor , { ( "*" | "/" ) , factor } ;
factor       = number | string | identifier | "(" , expression , ")" ;
```

La **precedencia está en la forma de la gramática**: `expression` está por encima de `term`, entonces `*` se agrupa antes que `+`.

Herramienta para explorar

AST Explorer — pegás código y ves el árbol.

👉 <https://astexplorer.net/>

Análisis semántico

¿tiene sentido lo que dice?

El parser tampoco valida todo

```
let x: number = "hola";  
println(y);
```

Los dos **parsean perfecto**: la gramática los acepta, el AST queda bien formado.

Pero el primero asigna un *string* a un `number`, y el segundo usa una variable que nunca se declaró.

La escalera de validación

Etapa	Rechaza
Análisis léxico	caracteres que no forman ningún <i>token</i>
Análisis sintáctico	<i>tokens</i> en un orden que la gramática no admite
Análisis semántico	árboles bien formados que no significan nada válido

Cada etapa confía en que la anterior hizo su trabajo, y deja lo que no le corresponde para la siguiente.

¿Dónde vive el chequeo?

PrintScript es **tipado**: alguien tiene que verificar que los tipos cierren.

- ¿Un **módulo aparte** que recorre el AST antes de ejecutar?
- ¿**Adentro del interpreter**, chequeando a medida que evalúa?

No hay una única respuesta. Pero hay una diferencia grande:
cuándo se entera el usuario del error.

Interpreter

del árbol al efecto

Recorrer y evaluar

Interpretar es **recorrer el árbol** y, en cada nodo, hacer lo que ese nodo significa.

```
Object evaluate(Node node, Environment env) {  
    return switch (node) {  
        case NumberLiteral n -> n.value();  
        case Identifier i    -> env.lookup(i.name());  
        case BinaryExpression b -> apply(b.operator(),  
                                          evaluate(b.left(), env), evaluate(b.right(), env));  
        // ...  
    };  
}
```

El estado vive afuera

El **Environment** guarda las variables. El árbol no cambia nunca.

```
let x: number = 5;    // env: { x -> 5 }  
let y: number = x+1; // env: { x -> 5, y -> 6 }
```

Árbol inmutable + estado explícito $\Rightarrow$ **testable**.

Un detalle que se paga caro

Un archivo de PrintScript puede ser **grande**.

Si el *lexer* devuelve `List<Token>` y el *parser* devuelve el AST completo, todo el programa vive en memoria a la vez.

Alternativa: que las etapas sean **lazy** (`Iterator` , *streams*) y procesen *statement* por *statement*.

A trabajar 🚀